

Thanos benchmark report

Interactive cross-filtering for R/Shiny — performance across scales, backends, and storage layouts

Thanos project (github.com/NathanSiemers/Thanos)

2026-08-05

Contents

What this report covers	1
1. The per-interaction data path (flights, 336,776 rows, in memory)	2
2. In-memory vs tall/skinny SQLite (flights)	3
3. 38 million rows: SQLite vs DuckDB in aggregate mode	3
4. Storage layouts: melt vs wide vs parquet-direct (DuckDB)	4
Overall guidance by scale	5

What this report covers

Thanos is a reusable R/Shiny module for interactive cross-filtering: the user picks columns, gets auto-generated filter widgets, and every selected column shows a live histogram of the rows passing all *other* filters, with that column's own selection overlaid. Every slider drag or checkbox click must therefore recompute masks and repaint plots — performance is the product.

This report collects the project's benchmarks in the order they were run, from a 337 thousand-row in-memory data frame to 38 million taxi rows in DuckDB. Each section explains **what is being measured and why it matters to interactivity**, then gives the numbers and their interpretation. All results come from the scripts in `bench/` and are reproducible:

```
Rscript bench/bench_masks.R      # the per-interaction data path
Rscript bench/bench_backends.R   # in-memory vs SQLite column store
Rscript bench/bench_big.R        # 38M rows: SQLite vs DuckDB aggregates
Rscript bench/bench_layouts.R    # melt vs wide vs parquet-direct
```

Test machine: R 4.5.3, Linux, 28 cores, 377 GB RAM (rstudio container). Times are means over repeated runs after a warm-up iteration; databases are on local disk.

A note on architecture, since it frames every number below: the module recomputes **only one column’s mask** when that column’s filter changes, combines all masks with cumulative ANDs ($O(3k)$ vector operations for k selected columns, not $O(k^2)$), and renders plots from **pre-binned counts** ($O(\text{bins})$, not $O(\text{rows})$). The benchmarks measure exactly these steps.

1. The per-interaction data path (flights, 336,776 rows, in memory)

What is measured. One user interaction — say, dragging the `dep_delay` slider — triggers this pipeline: recompute that column’s logical mask, re-combine the leave-one-out masks for every selected column, re-tabulate binned counts, and rebuild each histogram. If the whole pipeline stays well under ~150 ms per plot, the UI feels instant. `bench/bench_masks.R` times each stage in isolation on `nycflights13::flights`.

Why the “OLD” row exists. The pre-merge prototypes rebuilt each histogram by handing all ~337k raw values to `ggplot2::geom_histogram` on every interaction. The final row measures that legacy approach for contrast with the pre-binned design.

operation	time
make_mask numeric (dep_delay in [-10, 60])	10.7 ms
make_mask categorical (3 carriers)	22.2 ms
leave-one-out combine, $k = 3$ vars	15.8 ms
leave-one-out combine, $k = 6$ vars	44.4 ms
leave-one-out combine, $k = 10$ vars	57.8 ms
tabulate counts, numeric 50 bins	6.6 ms
plot_histo build (ggplot object), numeric	22.3 ms
plot_histo build (ggplot object), categorical	25.7 ms
plot_histo build + render to png, numeric	245 ms
OLD: geom_histogram over 337k raw rows (build + render)	920 ms

Interpretation.

- The *data* side of an interaction is cheap: one mask recompute (~10–20 ms) plus one leave-one-out combine (~16–58 ms depending on how many columns are selected) plus ~7 ms of counting per plot. Comfortably inside the interactivity budget.
- Plot **rendering** dominates: ~245 ms per histogram, of which only ~22 ms is building the ggplot object — the rest is fixed ggplot/grid/png-device overhead that does not grow with data size. Pre-binning made each render 3.7× faster than the legacy full-data approach (920 ms), and decoupled render cost from row count entirely.
- Future lever if many panels ever feel sluggish: a faster graphics device (ragg) or a base-graphics **barplot** re-implementation of the same visual. Not currently needed.
- Not visible in the table but architecturally decisive: the old module *also* tore down and rebuilt every filter widget on every input (a `renderUI` dependency bug) and re-filtered the full data frame once per plot. The merged module touches one mask and k plots, nothing else.

2. In-memory vs tall/skinny SQLite (flights)

What is measured. The same module can run against an in-memory data frame (`backend_memory`) or a SQLite database in the tall/skinny layout `long_data(row_id, column_name, value_num, value_txt)` with a precomputed `column_registry` (`backend_sqlite`). The module treats both as **column stores**: when the user selects a column it is fetched once, in full, and all filtering then happens in R on the cached vector. So the only structural difference between backends is the cost of that one-time fetch, plus the cost of the metadata needed to build widgets. `bench/bench_backends.R` measures both. Database: 6,352,149 long rows, 282 MB, indexed on (`column_name`, `row_id`).

operation	memory	sqlite
backend open (registry read)	—	340 ms
get_column_info, all 19 cols	322 ms (first scan)	2.2 ms (registry)
get_column(dep_delay)	~0 ms	203 ms
get_column(distance)	~0 ms	210 ms
get_column(carrier)	~0 ms	227 ms
get_column(dest)	~0 ms	231 ms
make_mask on fetched column	8.5 ms	8.5 ms (identical)

Interpretation.

- The project hypothesis (“might not be too different, because this isn’t a huge database”) is confirmed, and the benchmark shows *why*: after a ~200–230 ms one-time fetch per selected column, every per-interaction cost is byte-identical between backends, because the module filters cached vectors either way.
- The registry pays for itself immediately: widget metadata (ranges, levels, NA counts, distinct-value counts, robust quantiles) is ~146× faster from the registry than a first in-memory scan, because it was precomputed at build time.
- The (`column_name`, `row_id`) index is what keeps fetches at ~200 ms over a 6.4M-row long table; without it every fetch is a full scan.

3. 38 million rows: SQLite vs DuckDB in aggregate mode

What is measured. At taxi scale (NYC TLC yellow cab 2023: 38,310,226 rows, 13 columns kept, 460–494M cells in the melt) the column-store approach stops being reasonable — a single fetched column is ~300 MB, and R-side boolean algebra on 38M-long vectors costs seconds per interaction. The module therefore switches to **aggregate mode**: no column vector ever enters R; histogram counts, row counts, and the row mask are SQL queries composed from the current filter state, and only ≤ 50 bin counts cross the wire per plot. `bench/bench_big.R` runs those exact queries against the same tall/skinny schema built in both engines, with two filters active (`fare_amount` in $[5, 60]$, `payment_type` in $\{1, 2\}$) — a realistic mid-interaction state.

Build cost (12 parquet months, ~607 MB compressed):

step	SQLite (R melt via arrow/data.table)	DuckDB (all-SQL)
build	559 s	94 s
index	690 s	64 s
file size	27.8 GB (460M long rows*)	5.9 GB (494M long rows)

* The SQLite build initially missed `airport_fee` in months that name the column `Airport_fee`: R/arrow matches column names case-sensitively where DuckDB does not. The build script now normalizes the name; benchmark columns were unaffected.

Per-interaction aggregate queries:

operation	DuckDB	SQLite
get_binned numeric (GROUP BY over filtered rows)	477 ms	75.5 s
get_binned categorical	229 ms	50.7 s
get_count	361 ms	45.1 s
get_column full fetch (38M values)	3.3 s	28.3 s
get_row_mask	0.9 s	55.5 s

Interpretation.

- **DuckDB is the answer at this scale.** Sub-second aggregates make the module genuinely interactive on 38M rows: with the 500 ms debounce, a slider drag settles in roughly a second per histogram. Its columnar, parallel execution engine is well matched to the row-set semi-joins the tall/skinny layout requires.
- **SQLite does not survive this scale in aggregate mode.** Every query walks hundreds of millions of b-tree index entries on a single core; 45–75 s per query is two orders of magnitude off. This is not an indictment of SQLite generally — section 2 shows it is perfectly good as a column store up to roughly flights scale — it is the wrong engine for large analytical scans.
- The same module code drives both engines; the demo apps differ only in the backend constructor line. That was the point of the backend contract, and it is what made this an apples-to-apples comparison.

4. Storage layouts: melt vs wide vs parquet-direct (DuckDB)

What is measured. Section 3 fixed the engine question; this section asks whether the tall/skinny layout itself costs performance. All three variants run in the *same* DuckDB process over the *same* 38,310,226 rows, so the only variable is physical layout:

- **melt** — the production path: `long_data(row_id, column_name, value_num, value_txt)`. Because there are no real columns, each filter becomes a *semi-join*: `row_id NOT IN (SELECT row_id FROM long_data WHERE column_name = 'fare_amount' AND ...)` against the same 494M-row table. Column fetches ship `(row_id, value)` pairs — roughly twice the necessary bytes — and are reassembled into full-length vectors in R (which is also how NA-by-absence is reconstructed).

- **wide** — one real column per variable (the **wide** table the DuckDB build materializes anyway as an intermediate). Filters are ordinary `WHERE fare_amount BETWEEN 5 AND 60` predicates: no joins. Fetches ship just the ordered column.
- **parquet-direct** — the same expressions evaluated straight over `read_parquet('yellow_tripdata_*.parquet')` with **no database build at all**. The one capability it lacks: parquet scan order is not guaranteed stable across queries, so there is no trustworthy row ID — histogram counts and row counts are exact, but the `rows()/mask()` pointer a parent app consumes would require materializing row IDs first (at which point you have built the wide table).

Same interaction state as section 3.

operation	melt	wide	parquet-direct
get_binned numeric, 2 filters	477 ms	168 ms	633 ms
get_binned categorical, 1 filter	233 ms	58 ms	419 ms
get_count, 2 filters	358 ms	76 ms	402 ms
get_column full fetch	3.3 s	1.6 s	1.5 s
get_row_mask, 2 filters	819 ms	372 ms	n/a (no stable row_id)

Interpretation.

- **The melt tax is real but bounded: roughly 3–5× on aggregates.** Wide’s plain column predicates beat the melt’s self-semi-joins on every query (58–168 ms vs 233–477 ms). DuckDB absorbs the join work impressively — half a second over 494M long rows — but it cannot make a join as cheap as no join.
- **Parquet-direct is remarkable for what it doesn’t need.** With zero build time, zero database file, and ~600 MB of compressed source data, it lands in the same class as the melt (it pays decompression and file scanning per query instead of join work). For exploratory “point Thanos at a directory of parquet and go” use, it is viable today for histograms and counts.
- **Practical guidance.** The tall/skinny melt remains the right *interchange* schema — it is what lets one `backend_dbi` implementation serve SQLite and DuckDB identically, matches the project’s relational-modeling goal, and its absolute numbers are interactive. When maximum speed matters, a wide-table DuckDB backend is a drop-in win (the backend contract hides the layout from the module entirely); the build even materializes the wide table already. A parquet-direct backend would additionally need row-ID materialization to support the parent-app row pointer.

Overall guidance by scale

data size	recommended setup	expected feel
$\leq \sim 1\text{M}$ rows	<code>backend_memory</code> , vector mode	instant
$\sim 1\text{M}$ rows, data in a DB	<code>backend_sqlite</code> column store, vector mode	instant after ~ 0.2 s per column selection

data size	recommended setup	expected feel
tens of millions of rows	backend_duckdb (melt), aggregate mode	~0.5–1 s per interaction
same, maximum speed	wide-table DuckDB backend (future drop-in)	~0.1–0.2 s per interaction
ad-hoc parquet exploration	parquet-direct backend (future; no row pointer yet)	~0.5–1 s per interaction, zero build